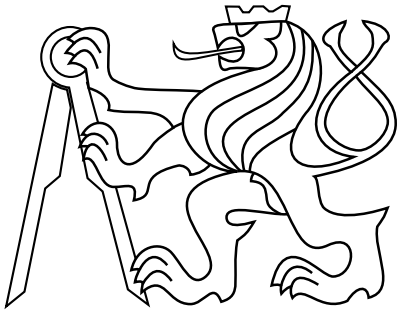
Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics
Multi-robot Systems



Learning an Autonomous Drone Racing Policy via Differentiable Simulation

Master's Thesis

Adam El Ghaib

Prague, January 2026

Study programme: Cybernetics and Robotics (Space Master)

Supervisor: Ing. Robert Pěnička, Ph.D.

Abstract

Todo

Keywords Unmanned Aerial Vehicles, Autonomous Drone Racing, Simulation to Reality Transfer, Reinforcement Learning, Domain Randomization

Abbreviations

UAV Unmanned Aerial Vehicle

DR Domain Randomization

Sim2Real Simulation-to-Reality

CTBR Collective Thrust and Body Rates

ADR Autonomous Drone Racing

OCP Optimal Control Problem

OCPs Optimal Control Problems

PMP Pontryagin's Maximum Principle

SRT Single Rotor Thrust

CTBR Collective Thrust and Body Rates

LVYR Linear Velocities and Yaw Rate

Contents

1 Introduction

1.1 Background

Give context about the problem

What is ADR?

- Navigate through a sequence of gates as fast as possible within the platform capabilities...

What are the challenges of ADR?

- **Where am I?** → Perception
  challenges: limited sensory data, noisy and uncertain partial observability
- **Where do I go?** → Planning
  safe efficient path planning
- **How do I get there?** →Control
  Handle the non-linear underactuated dynamics to execute the planned motion under actuator limits and model uncertainties

1.2 Related Work

Present the state of the art, how do the different research labs approach this problem

Open Questions

- How to improve sample efficency
- Generalization to multiple tasks (how to race while avoiding obstacles, how to race agains other drones)
- Transferability of the conterllers under unmodeled effects

1.3 Structure and Objectives

2 Preliminaries

This chapter covers important theoretical aspects on which this thesis is constructed. Section 2.1 formulates the Optimal Control Problem (OCP) of drone racing and presents a theoretical solution based on the application of Pontryagin's Maximum Principle (PMP). The optimal solution derived from PMP serves primarily as an upper theoretical bound, as many of its underlying assumptions do not hold in real-world scenarios. Practical implementations must account for model uncertainties, disturbances, and system constraints that deviate from the idealized formulation.

In this context, Sections 2.3 and 2.4 review the state-of-the-art control approaches used to achieve high-performance drone racing, covering both classical control techniques and modern learning-based methods.

2.1 Optimal Control Problem

cite hurak, or well...some OC textbook

General Discrete-Time OCP

In general, a nonlinear discrete-time OCP can be formulated as the optimization of a running cost (or loss) function $L_k()$, plus a terminal cost function $\phi()$, subject to the system's dynamics $\mathbf{f}_k()$, and the set of permitted states $\mathcal{X}_k$, and controls $\mathcal{U}_k$.

$$\begin{aligned} \underset{\mathbf{X},\,\mathbf{U}}{\text{minimize}} \quad & \left(\phi(\boldsymbol{x}_N, N) + \sum_{k=0}^{N-1} L_k(\boldsymbol{x}_k, \boldsymbol{u}_k) \right) \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = \mathbf{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{u}_k \in \mathcal{U}_k, \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}_k, \quad k = 0, \ldots, N, \end{aligned} \tag{2.1}$$

Where

N is the final discrete time,
$\boldsymbol{x}_k \in \mathbb{R}^n$ is the state at the discrete time $k \in \mathbb{Z}$,
$\boldsymbol{u}_k \in \mathbb{R}^m$ is the control at the discrete time $k \in \mathbb{Z}$,
$\mathbf{X} = [\boldsymbol{x}_0, \ldots, \boldsymbol{x}_N]$ is the trajectory of states,
$\mathbf{U} = [\boldsymbol{u}_0, \ldots, \boldsymbol{u}_{N-1}]$ is the trajectory of control actions.

Minimum-Time OCP

This last formulation of an OCP can be further specified for the case of drone racing in several manners. Since the main objective is to complete a lap through the racing track as fast as possible, this problem can be written as minimum-time problem.

$$\begin{aligned} \underset{\mathbf{U},\, N}{\text{minimize}} \quad & \sum_{k=0}^{N-1} 1 \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = \mathbf{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{u}_k \in \mathcal{U}_k, \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}_k, \quad k = 0, \ldots, N, \end{aligned} \tag{2.2}$$

The racing track restrictions, i.e. passing through a sequence of gates while avoiding obstacles, can be introduced as hard constraints under the set of permitted states $\mathcal{X}_k$ as

$$\mathcal{X}_k = \left\{ \boldsymbol{x}_k \in \mathbb{R}^n \;\middle|\; \boldsymbol{p}_k \in \mathcal{G}_{g(k)}, \quad \boldsymbol{p}_k \notin \mathcal{O}_j, \quad \forall\, j \in \{1, \ldots, M\} \right\}, \tag{2.3}$$

where

$\boldsymbol{p}_k \in \mathbb{R}^3$ is the position of the drone at time step k,
$\mathcal{G}_{g(k)}$ is the feasible region defined by gate g at time step k, with $g(k) \in \{1, \ldots, G\}$ denoting the index of the gate to be passed at k,
G is the total number of gates in the racing track,
$\mathcal{O}_j$ is the region occupied by the j-th obstacle,
M is the total number of obstacles.

Simplified Minimum-Time OCP

The anterior problem formulation (subsection 2.1.2), is rather complex. It has to solve simultaneously for an optimal trajectory that passes through all the gates and avoids obstacles. And, find the optimal control sequence achivable by the drone given its dynamics and actuator constrains. To simplify this problem, throughout this thesis, either an optimal trajectory or an optimal path will be assumed to be given by an external source. Thereafter, the Optimal Control Problems (OCPs) that we adress are

Trajectory Tracking OCP

$$\begin{aligned} \underset{\mathbf{U}}{\text{minimize}} \quad & \sum_{k=0}^{N-1} \left\| \boldsymbol{x}_k - \boldsymbol{x}_k^{\text{ref}} \right\|_{\mathbf{Q}}^2 \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = \mathbf{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{u}_k \in \mathcal{U}_k, \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}_k, \quad k = 0, \ldots, N, \end{aligned} \tag{2.4}$$

Path Progress OCP

2.2 Pontryagin's Maximum Principle

maybe solve it using a pmm and Pontryagin's Maximum Principle

Since it is impossible to perfectly model the state transition equation, and the state estimation is not perfect, the optimal solution breaks under real world conditions.

- **2.3 Classical Control approaches**
- **Differential Flatness Based Control**
- **Model Predictive Control**
- **2.4 Learning-Based approaches**
- **Model-Free Reinforcement Learning**
- **Model-Based Reinforcement Learning**

3 Methodology

This chapter presents the methodology used in this thesis to train the neural network policies. In section 3.1, we adress the core elements of the simulation architecture, define the quadrotor model, and explain how the Simulation-to-Reality (Sim2Real) gap is minimized by using Domain Randomization (DR). Furthermore, we introduce the implemented control abstraction levels. Section 3.2, outlines the training objectives used for Autonomous Drone Racing (ADR). Lastly, section 3.3 details how differentiability is leveraged to optimize the policies in order to minimize the training task's cost function.

3.1 Simulation Framework

Insert cool diagram

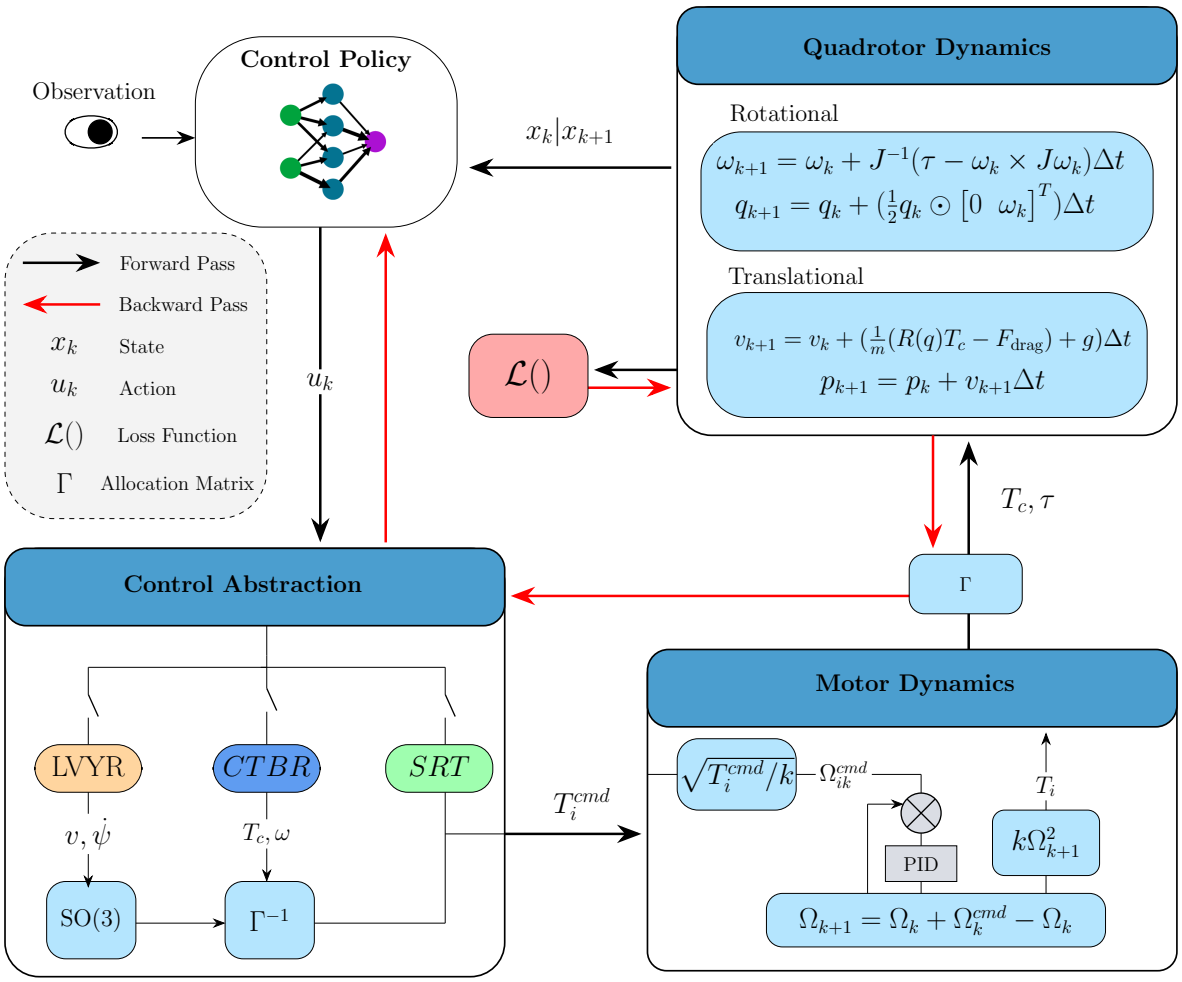


Figure 3.1: Your Caption

- **Quadrotor Model**
- **Domain Randomization**
- **Control Abstraction**

The control abstraction refers to the modality of the policy outputs used to control the quadrotor. The choice of control abstraction affects both the performance of the controller for a given task and the sample efficiency during training. For this reason, it is worthwhile to investigate and implement multiple control abstractions.

In this thesis, three control modalities are considered. The policy output layer employs a sigmoid activation function for all control modalities, ensuring that the action space is bounded, which facilitates stable learning. The lowest level of abstraction consists of directly outputting Single Rotor Thrust (SRT) for each motor. An intermediate level of abstraction outputs the collective thrust and the desired body rates, which are subsequently mapped to individual rotor thrusts through a control allocation matrix. The highest level of abstraction outputs desired linear velocities and yaw rate, which are converted into a wrench using an SO(3) geometric controller TODO : Cite Lee, and subsequently mapped to rotor thrusts in a manner analogous to the collective thrust and body rate formulation. The mathematical formulation of these transformations is presented next.

Single Rotor Thrust

When the SRT control mode is choosen, the only transformation to be applied is scaling the policy output $\boldsymbol{u} \in [0, 1] \in \mathbb{R}^4$ to the maximum thrust that can be produced by the motors, T_{max}. Therefore, the transformation is as simple as:

$$\boldsymbol{T}_{\text{cmd}} = \boldsymbol{u} \cdot T_{\text{max}} \tag{3.1}$$

TODO : Add min thrust mapping xd

Once the scaling is applied, to better emulate the behaviour of a real motor, a first order model with rotor speed saturation is implemented. First, the commanded motor thrust $\boldsymbol{T}_{\text{cmd}}$ are converted to commaned rotor speed. This mapping can be obtained experimentally by fitting a quadratic curve to the experimental data. In most cases the following mapping apprixmation is good enough.

$$\boldsymbol{T}_{\text{cmd}} \approx k\boldsymbol{\Omega}^2_{\text{cmd}} \tag{3.2}$$

Then the commanded rotor speeds $\boldsymbol{\Omega}_{\text{cmd}}$, are passed through the following first order motor model Equation 3.3, and saturated according to the maximum rotor spee rate, $\dot{\Omega}_{\text{cmd}}$ given by the manufacturer.

$$H(s) = \frac{1}{k_m + s} \tag{3.3}$$

Finally, the rotor speeds obtained after the described process are mapped again to single rotor thurst values using Equation 3.2 and then converted to collective force and torques with Equation 3.4, to calculate the next state through the quadrotor dynamics model.

$$\begin{bmatrix} T_c \\ \tau_1 \\ \tau_2 \\ \tau_3 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \tag{3.4}$$

Modeling the motor dynamics presents more realistic data which the policy will encounter at deployement. However, for training purposes it has been decided to skip the motor dynamics during the backporopagation pass as it significantly destabilizes the gradients computation. Particularly, the mapping of Equation 3.2 produces high variations on the gradient, degrading the training performance and sample efficiency.

Collective Thrust and Body Rates

When Collective Thrust and Body Rates (CTBR) control mode is selected, the normalized outputs of the policy $\boldsymbol{u} \in [0, 1] \in \mathbb{R}^4$, are mapped to collective thrust and body rates. The scaling is as follows,

$$T_{c,\text{cmd}} = u_1 \cdot (T_{\max} - T_{\min}) + T_{\min} \tag{3.5}$$

$$\omega_{i,\text{cmd}} = (2u_{i+1} - 1) \cdot \omega_{i,\max}, \quad i \in \{1, 2, 3\} \tag{3.6}$$

where $\omega_{i,\max}$ indicates the maximum body rate of the respective axis, (roll, pitch, yaw). Once the rates are calculated, they can be converted to torque values,

$$\boldsymbol{\tau}_{\text{cmd}} = k_p \cdot \frac{(\boldsymbol{\omega}_{\text{cmd}} - \boldsymbol{\omega})}{dt} J \tag{3.7}$$

aditionally, a proportinal gain k_p can be introduced to adjust the stiffness of the conversion. Given the commanded collective thrust and torques, the motor thursts can be obtained from Equation 3.4 and then the same procudeure of subsection 3.1.3 is applied during the forward and backward passes.

Linear Velocities and Yaw Rate

Lastly, the highest level of abstraction that is implement consists on commanding Linear Velocities and Yaw Rate (LVYR). With this control mode, the policy outputs are scaled as,

$$v_{i,\text{cmd}} = (2u_i - 1) \cdot v_{i,\max}, \quad i \in \{1, 2, 3\} \tag{3.8}$$

$$\omega_{3,\text{cmd}} = \dot{\psi}_{\text{cmd}} = (2u_4 - 1) \cdot \dot{\psi}_{\max} \tag{3.9}$$

where ψ denotes the yaw angle and $v_{i,\text{cmd}}$ represents the commanded linear velocity along each body axis. Mapping these control commands to thrust commands is less straightforward than in the previous two methods. Instead, the collective thrust and torques are obtained by applying a transformation in the Special Orthogonal group SO(3). The transfomations proceed in the following manner.

First, the acceleration vector is computed as

$$\mathbf{a}_{\text{cmd}} = k_v \cdot (\mathbf{v}_{\text{cmd}} - \mathbf{v}) + g\hat{\mathbf{b}}_3 \tag{3.10}$$

where k_v is a proportional gain on the velocity. Note that the acceleration along the third body axis is compensated for gravity; $\hat{\mathbf{b}}_3$ denotes the unit vector along this axis. Then the commanded collective thrust is the Unmanned Aerial Vehicle (UAV)'s mass times the acceleration vector.

TODO Correct this...

$$T_{c,\text{cmd}} = m\mathbf{a}_{\text{cmd}} \tag{3.11}$$

TODO Double check wether projecting the heading angle is actually correct

To obtain the commanded bodytorques, the heading vector is obtained by projecting the yaw angle into the world plane defined by $\text{span}\{\hat{e}_1, \hat{e}_2\}$. The heading vector, serves an auxiliary vector to compute $\hat{\mathbf{b}}_2$ and $\hat{\mathbf{b}}_1$.

$$\mathbf{h} = \begin{bmatrix}\cos\psi & \sin\psi & 0\end{bmatrix}^T \tag{3.12}$$

$$\hat{\mathbf{b}}_2 = \frac{\hat{\mathbf{b}}_3 \times \mathbf{h}}{\|\hat{\mathbf{b}}_3 \times \mathbf{h}\|} \tag{3.13}$$

$$\hat{\mathbf{b}}_1 = \hat{\mathbf{b}}_2 \times \hat{\mathbf{b}}_3 \tag{3.14}$$

In this way, the commanded orientation is defined by $\mathbf{R}_{\text{cmd}} = \begin{bmatrix}\hat{\mathbf{b}}_1 & \hat{\mathbf{b}}_2 & \hat{\mathbf{b}}_3\end{bmatrix}$ and the error between the commanded orientation and the current orientation is computed as in TODO Cite Lee.

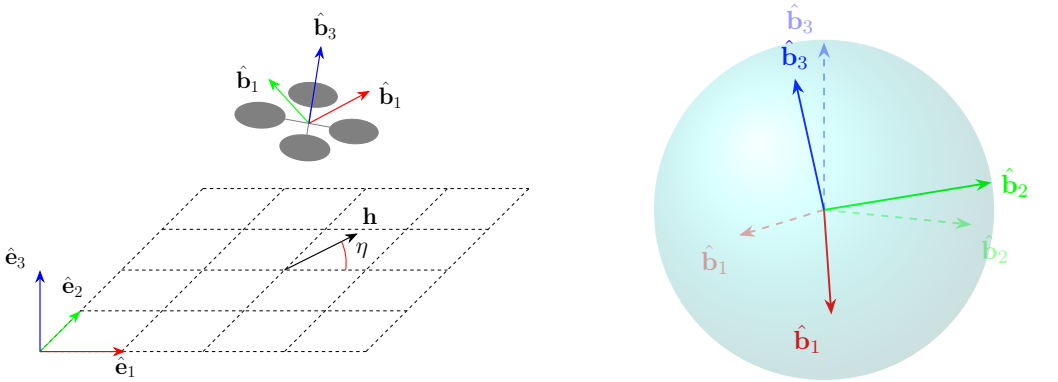
$$\mathbf{e}_R = \frac{1}{2}\left(\mathbf{R}_{\text{cmd}}^T\mathbf{R} - \mathbf{R}^T\mathbf{R}_{\text{cmd}}\right)^\vee \tag{3.15}$$

The complete derivation of Equation 3.15 can be found in the original paper, [**Lee**]. To provide intuition, note that if $\mathbf{R}_{\text{cmd}} = \mathbf{R}$, then $\mathbf{R}_{\text{cmd}}^T\mathbf{R} = \mathbf{I}$ and the error is zero. Otherwise, the matrix $\mathbf{R}_{\text{cmd}}^T\mathbf{R}$ represents the relative rotation between the current and desired orientations. The skew-symmetric part of this matrix, given by $\frac{1}{2}(\mathbf{R}_{\text{cmd}}^T\mathbf{R} - \mathbf{R}^T\mathbf{R}_{\text{cmd}})$, encodes the axis and magnitude of the rotation error. The vee map $(\cdot)^\vee$ then converts this skew-symmetric matrix into a vector in $\mathbb{R}^3$, corresponding to the rotation error expressed in the Lie algebra $\mathfrak{so}(3)$, that is to say, $^\vee\colon \mathfrak{so}(3) \to \mathbb{R}^3$. Then, the factor $\frac{1}{2}$ ensures consistency with the standard definition of the vee operator and avoids double counting of the off-diagonal terms.

Adittionally, an error on the angular velocity of the rotation is introduced as,

$$\mathbf{e}_{\boldsymbol{\omega}} = \boldsymbol{\omega} - \mathbf{R}^T\mathbf{R}_{\text{cmd}}\boldsymbol{\omega}_{\text{cmd}} \tag{3.16}$$

where $\boldsymbol{\omega}_{\text{cmd}} = \begin{bmatrix}0 & 0 & \dot{\psi}_{\text{cmd}}\end{bmatrix}^T$. Note that the error between the desired rotational velocity and the current rotational velocity line on different tangent planes to the unit sphere, $\dot{\mathbf{R}} \in \text{T}_{\mathbf{R}}SO(3)$ and $\dot{\mathbf{R}}_{\text{cmd}} \in \text{T}_{\mathbf{R}_{\text{cmd}}}SO(3)$. For this reason, as explained in [**Lee**], $\dot{\mathbf{R}}_{\text{cmd}}$ is transformed into a vector in the $\text{T}_{\mathbf{R}}SO(3)$ space to be able to compare it to $\dot{\mathbf{R}}$ and obtain the error;Equation 3.16.



(a) Projection of the drone's yaw angle ψ.

(b) Representation of the $SO(3)$ rotation space.

Figure 3.2: Geometric interpretation of Equations 3.12–3.15

3.2 Training Tasks

Trajectory Tracking

Path Progress

3.3 Differentiable Optimization

4 Results

5 Conclusion

6 Future Directions

7 References

A Appendix A